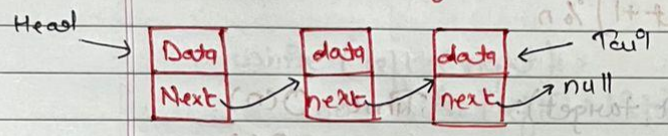
Linked list

A linked list is a linear data structure where elements (called nodes) are connect using pointers. Unlike arrays, elements are not stored in contiguous memory locations

Each node in a linkedlist typically contains:

1. Data → the actual value
2. Next → a reference to the next node



- Head is a reference to the first and tail is a reference to the last node in the linkedlist.

```
public static class Node {
        int data;
        Node next;

        public Node(int data) {
                this.data = data;
                this.next = null;
        }
}

public static Node head;

public static Node tail;

public static int size;
```

Q. Where it is used?

1. Music / Video Playlists

Example playlist: Song1 → Song2 → Song3 → Song4

Next song: current = current.next

Media players internally use structures similar to linked lists.

2. Undo/Redo Systems

Example: State1 → State2 → State3 → State4

Undo moves backward.

Often implemented with **doubly linked lists**.

Used in:

- Text editors
- Photoshop
- IDEs

Q. Memory space used?

1. Memory Used by One Node

A singly linked list node usually looks like this:

```
class Node {
   int data;
   Node next;
}
```

Each node stores **two things**:

1. **data**
2. **reference (pointer) to next node**

So memory per node = memory(data) + memory(pointer)

2. Actual Memory Size (Typical 64-bit JVM)

Approximate values:

Component	Memory
int data	4 bytes
reference (next pointer)	8 bytes

→ Add First in LL

```
public void addFirst (int data) {
    Node newNode = new Node(data);
    size ++;

    if(head == null) {
        head = newNode;
        tail = newNode;
        return
    }

    newNode.next = head
    head = newNode
}
```

→ creates a new Node

→ If list is empty → both head and tail point to the new node

→ Connect new node to the current head.

→ Update head to new node.

Time: O(1)

→ Add last in LL

```
public void addlast (int data) {
    Node newNode = new Node(data);
    size ++;

    if(head == null) {
        head = newNode;
        tail = newNode;
        return
    }

    tail.next = newNode
    tail = newNode
}
```

→ Connect old tail to the new node.

→ Update tail to the new node.

Time: O(1)

→ Print a LL

```
public void printLinkedList() {
    Node temp = head;
    while (temp != null) {
        sys (temp.data)
        temp = temp.next
    }
}
```

→ A temp pointer temp starts at head

→ print current node's data, move temp to the next node.

Time: O(n)

→ Add In a middle / specific Index

```
public void addInMiddle (data, int idx) {
    if (idx == 0) {
        addFirst (data)
        return
    }

    Node newNode = new Node (data)
    size++;
    Node temp = head;
    int i=0;

    while ( i < idx-1) {
        temp = temp.next
        i++
    }

    newNode.next = temp.next
    temp.next = newNode
}
```

→ Fixed at the start

→ start from head and move forward until reaching the node just before the desired position (idx-1)

Time: O(n) → Traversal is required up to the index before insertion.

→ Remove First Node

```
public void remove First() {
        if (size == 0) {
            sys("list is empty")
            return
        } else if (size == 1) {
            head = null;
            tail = null;
            size --
            return
        }

        head = head.next
        size --
}
```

→ check if list is empty

→ If there is only one node: Set both head and tail to null.

Otherwise, move the head to the next node

Time: O(1)

→ Remove last Node()

```
public int removelast() {
        if(size == 0) {
            sys ("list is empty")
            return Integer.MIN_VALUE
        } else if(size == 1) {
            int val = tail.data
            head = tail = null
            size --
            return val
        }

        Node prev = head
        for(int i=0; i<size-2; i++) {
            prev = prev.next
        }
```

→ Traverse till the second last (i<size-2)

```
int val = tail.data;
prev.next = null;   detach the last node
tail = prev;        update tail
size--;
return val;
}
```

Time: O(n)

→ Search (Iterative)

Search for a key in a linked list. Return the position where it is found. If not found, return -1.

```
public int iterativeSearch(int key){
    Node temp = head
    int index

    while (temp != null){
        if(temp.data == key){
            return index;
        }

        temp = temp.next
        index++;
    }

    return -1
}
```

Time: O(n)

Space: O(1)

(Recursive)

```
public int recursiveSearch (Node head, int key){
        if (head == null){
                return -1;
        }
        if (head.data == key){
                return 0;
        }
        int indexInRest = recursiveSearch (head.next, key);
        if (indexInRest == -1){
                return -1;
        }
        return indexInRest +1;
}
```

Time: O(n)

Space: O(n) → due to recursion stack

→ Reverse a linked list

```
public void reverse (){
    Node prev = null;        → keeps track of the reversed part
    Node current = tail = head;   → current node being processed.
                                  → because after reversal, the old head
                                    becomes the new tail.
    Node next;

    while (current != null){
        next = current.next;      → store the next node
        current.next = prev;      → reverse the link
        prev = current;
        current = next;           } Moving pointers forward
    }
    head = prev  → points to the last processed node (which becomes
                   the new head).
}
```

Time: O(n)